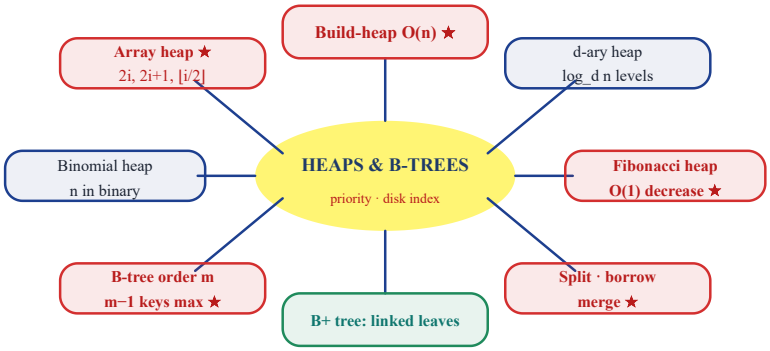
Advanced Data Structures: Heaps, Fibonacci Heaps, B-Trees

Binary heap · priority queue · d-ary & binomial heaps · Fibonacci heap · B-tree · B+ tree: exam-ready in eleven sheets

Eleven sheets: the array heap and its index formulas, every heap operation with its cost, why build-heap is linear, the heap family up to the Fibonacci heap, then the disk-friendly B-tree, its insert/delete rules and the B+ tree that every database index uses.



↑ the whole topic: red = high-yield, green = the database link

Syllabus coverage

The syllabus line for Data Structures and Algorithms reads: "**Internal data representation; Abstract data types; Elementary data structures: arrays, lists, stacks, queues, trees, graphs; Advanced data Structures: heaps, Fibonacci heaps, B-trees;**" This set takes the **advanced** clause in full, plus the B+ tree that every exam pairs with it. The ADTs and elementary structures are the chapter's first lesson; **heapsort**, searching and hashing belong to the sorting lesson, and **Dijkstra / Prim** to the graph-algorithms lesson — they appear here only where a heap is the reason for their running time.

High-yield list

① Heap = complete tree + heap order
② 1-based: 2i, 2i+1, parent ⌊i/2⌋
③ 0-based: 2i+1, 2i+2, ⌊(i−1)/2⌋
④ Insert / extract: O(log n)
⑤ **Build-heap: O(n) ★**
⑥ Heap search: O(n)
⑦ Fib heap decrease-key O(1) am.
⑧ Dijkstra + Fib: O(E + V log V)
⑨ Order m: ≤ m−1 keys, ≤ m children
⑩ Non-root ≥ ⌈m/2⌉ children ★
⑪ B-tree grows at the root
⑫ B+ tree: data in linked leaves

Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red** = exam trap · **green** = worked example

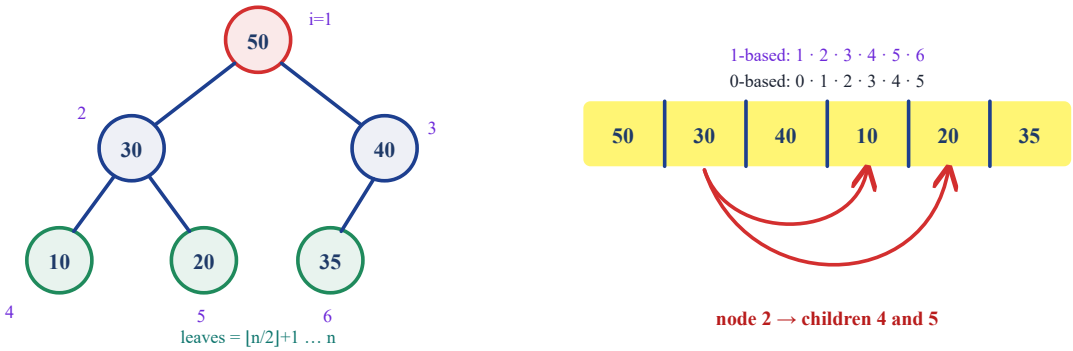
Contents

✓ Binary heap, array formulas, d-ary … p. 2
✓ Heap operations, sift-up/down … p. 3
✓ Build-heap O(n), heapsort, PQ … p. 4
✓ Binomial & Fibonacci heaps … p. 5
✓ B-tree: order m vs degree t … p. 7
✓ B-tree insert & delete … p. 8
✓ B+ tree & the big comparison … p. 9
✓ Night-before sheet … p. 10

✓ Heap costs compared, Dijkstra … p. 6 ✓ Practice MCQs + written … p. 11

1 The Binary Heap: a Complete Tree Stored in an Array

Definition: A **binary heap** is a binary tree that satisfies **two** properties at once. **Shape:** it is a **complete binary tree** — every level is full except possibly the last, which is filled **from the left**. **Order (heap property):** in a **max-heap** every parent ≥ its children, so the **maximum is at the root**; in a **min-heap** every parent ≤ its children, so the minimum is at the root. Nothing is said about left versus right child — a heap is **not** a binary search tree.



the same max-heap as a tree and as a level-order array — no pointers are stored

Quantity	Root at index 1 (CLRS)	Root at index 0 (C / Java arrays)
Left child of i	`2i`	`2i + 1`
Right child of i	`2i + 1`	`2i + 2`
Parent of i	`⌊i / 2⌋`	`⌊(i - 1) / 2⌋`
Leaves	indices `⌊n/2⌋ + 1 … n`	indices `⌊n/2⌋ … n - 1`
Last internal node	`⌊n/2⌋`	`⌊n/2⌋ - 1`

Shape facts ✍ the numbers: Height of an n-node heap = $\lfloor \log_2 n \rfloor$. A heap of height h holds between `2^h` and `2^(h+1) - 1` nodes. There are `⌈n/2⌉` leaves, and at most `⌈n / 2^(h+1)⌉` nodes of height h. Because the tree is complete, the array has **no gaps** — that is the only reason the index arithmetic works.

Memory trick: "**Double to go down, halve to go up.**" With the root at 1, a left child is the parent doubled and the right child is one more; the parent is the child halved and floored. For a 0-based array "**shift by one**": add 1 before you double, subtract 1 before you halve.

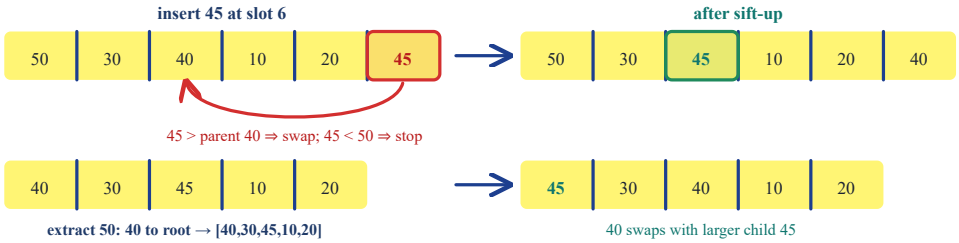
Exam trap ✍ three: **1.** Read the question's indexing first: `2i, 2i+1` is the answer **only** for a root at index 1; for a root at 0 it is `2i+1, 2i+2`. **2.** In a max-heap the **minimum** is in one of the **leaves** (positions ⌊n/2⌋+1 … n), not necessarily the last array cell. **3.** An array sorted in **descending** order is always a valid max-heap, but a valid max-heap is **not** in general sorted.

Generalisation: the d-ary heap: Same complete-tree-in-an-array idea, but every node has **d** children. 0-based: children of i are `d·i + 1 … d·i + d`, parent is `⌊(i - 1)/d⌋`. Height ≈ **log_d n**, so **insert / decrease-key = O(log_d n)** (fewer levels to climb) but **extract-min = O(d · log_d n)** (d children compared per level). A large d suits many decrease-keys and few extracts — e.g. Dijkstra on a dense graph.

2 Heap Operations: Sift-Up, Sift-Down and What Each Costs

Insert = sift-up (bubble-up): Put the new key in the **next free slot** (index n+1) so the tree stays complete, then **swap it with its parent while it is larger** (max-heap). At most one swap per level ⇒ **O(log n)**. **Increase-key** in a max-heap (decrease-key in a min-heap) is the same sift-up.

Extract-max = sift-down (heapify): Save the root, **move the last element to the root**, shrink n, then **swap it with its larger child** until both children are smaller. Two comparisons per level ⇒ **O(log n)**. This sift-down is called **MAX-HEAPIFY**.



a worked insert (top) and a worked extract-max (bottom) on the same array

Operation (max-heap, n keys)	Time	How
Find-max / peek	**O(1)**	read `A[1]`
Insert	O(log n)	append, sift-up
Extract-max	O(log n)	last → root, sift-down
Increase-key (decrease-key in a min-heap)	O(log n)	change, sift-up
Delete at a known index	O(log n)	swap with last, sift up *or* down
MAX-HEAPIFY one node	**O(log n)**	one sift-down (O(h) for a node of height h)
BUILD-HEAP of n keys	**O(n)**	heapify ⌊n/2⌋ … 1 (next sheet)
Search for an arbitrary key	O(n)	no order between siblings
Merge two heaps	O(n)	concatenate arrays, build-heap

```
MAX-HEAPIFY(A, i, n):   // 1-based
  l = 2i ; r = 2i + 1 ; big = i
  if l ≤ n and A[l] > A[big]: big = l
  if r ≤ n and A[r] > A[big]: big = r
  if big ≠ i:
      swap A[i], A[big]
      MAX-HEAPIFY(A, big, n)
```

Exam trap ✍ heapify vs build-heap: "Heapify takes O(n)" is a classic muddle. **HEAPIFY** (one sift-down from one node) is **O(log n)**. **BUILD-HEAP** (heapify every internal node, bottom-up) is **O(n)**. Read which one the question means. Also: a heap gives the max in O(1), but **searching** for any other key is O(n).

3 Build-Heap in O(n), Heapsort, and the Priority Queue

BUILD-MAX-HEAP: Copy the n keys into the array as they are, then call `MAX-HEAPIFY(A, i)` for **i = ⌊n/2⌋ down to 1**. Leaves (⌊n/2⌋+1 … n) are already one-node heaps, so they are skipped; going bottom-up guarantees both subtrees of i are heaps when i is heapified.

Why O(n), not O(n log n) ✍ 4 marks: Heapify at a node of height h costs **O(h)**, and there are at most `⌈n / 2^(h+1)⌉` such nodes. So
`T(n) = Σ_(h=0..⌊log n⌋) ⌈n/2^(h+1)⌉ · O(h) = O(n · Σ h/2^h)`.
Since `Σ_(h≥0) h/2^h = 2`, **T(n) = O(2n) = O(n)**.
Intuition: **half the nodes are leaves** and do no work, a quarter sink at most one level, an eighth at most two — the expensive nodes are rare.

Worked example ✍: Build a max-heap from `[3, 9, 2, 1, 4, 5]`, n = 6, so start at `⌊6/2⌋ = 3`.
i = 3: 2 vs child 5 → swap → `[3, 9, 5, 1, 4, 2]`
i = 2: 9 vs 1, 4 → no swap
i = 1: 3 vs 9, 5 → swap with 9 → `[9, 3, 5, 1, 4, 2]`; 3 now at 2, vs 1, 4 → swap with 4 → **`[9, 4, 5, 1, 3, 2]`**
Check: 9 ≥ 4, 5 · 4 ≥ 1, 3 · 5 ≥ 2 ✓ — **3 swaps**.

Exam trap ✍ two ways to build: Inserting the n keys **one by one** (top-down, sift-up each) costs **O(n log n)** in the worst case — e.g. keys arriving in increasing order into a max-heap, each rising to the root. The bottom-up **BUILD-HEAP is O(n)**. Both give a valid heap, but **not necessarily the same array**; a question that asks for "the heap after building" means the bottom-up method unless it says "insert one at a time".

Heapsort — the link to sorting: Build a max-heap, then repeat n − 1 times: swap `A[1] ⇔ A[k]`, shrink the heap by one, `MAX-HEAPIFY(A, 1)`. The array is a heap on the left and a growing sorted run on the right. Build-heap O(n) + (n − 1) extract-max at O(log n) each ⇒ **O(n log n) in best, average and worst case**. **In-place** (O(1) extra space) but **not stable**. A max-heap sorts ascending. The full comparison with quicksort and merge sort is in the sorting lesson of this chapter.

Priority queue (ADT): A collection where each element has a **priority** and the operations are `insert`, `find-max`, `extract-max`, `increase-key` (max-PQ) or their min versions. The heap is its **standard implementation**: an unsorted array gives O(1) insert but O(n) extract; a sorted array the reverse; a heap gives O(log n) for both.

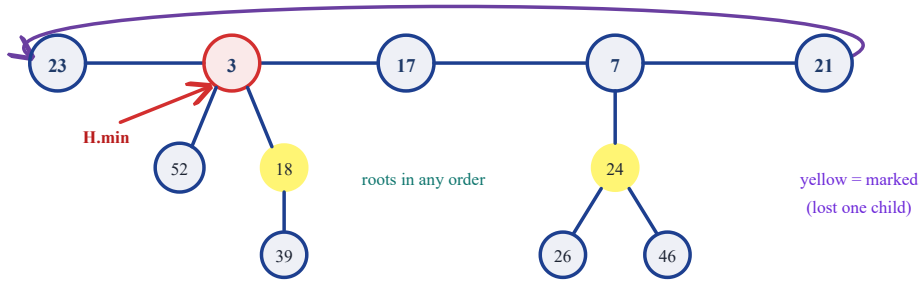
Where heaps / priority queues are used	Heap operation that matters
CPU / job scheduling, event-driven simulation	extract-min of the next deadline or event time
Dijkstra, Prim (graph-algorithms lesson)	extract-min once per vertex, **decrease-key** once per edge
Huffman coding, k-way merge, top-k of n items	repeated extract-min; a size-k heap gives top-k in **O(n log k)**

4 Mergeable Heaps: the Binomial Heap and the Fibonacci Heap

Binomial heap: **B_0** is one node; **B_k = two B_{k-1}, one root hung under the other**. B_k has **2^k nodes**, **height k**, **C(k, i)** nodes at depth i and a root of **degree k**. A binomial heap is a list of heap-ordered binomial trees with **at most one B_k per k** — the trees match the 1-bits of n: $13 = 1101_2 \Rightarrow B_3, B_2, B_0$, so at most **$\lfloor \log_2 n \rfloor + 1$ trees**.

Memory trick: "**A binomial heap is n written in binary; union is binary addition.**" Two B_k are two 1-bits in one column: link them into a B_{k+1} and **carry** — union is O(log n). Insert is "add 1": a long carry chain makes its worst case O(log n), but like a binary counter it is **O(1) amortized**.

Fibonacci heap: (Fredman & Tarjan, 1984) A collection of **heap-ordered trees** whose roots sit in a **circular, doubly-linked root list**, with a pointer **H.min** to the smallest root. The trees need **not** be binomial and several may share a degree — the structure is **lazy**: it postpones tidying until an extract-min forces it. Each node stores its **key, degree, parent, one child pointer, left/right sibling pointers** (children form their own circular list, so any splice or list concatenation is O(1)) and a boolean **mark** = "has lost a child since it last became a child of another node".



five trees in the root list; 23, 17 and 21 all have degree 0 — repeated degrees are allowed

The cheap operations: **Insert:** make a one-node tree, splice it into the root list, update H.min — **O(1)**, no tidying. **Union:** concatenate the two root lists, keep the smaller min — **O(1)**. **Find-min:** read H.min — **O(1)**.

Extract-min = the cleanup: Remove H.min, move its children to the root list, then **consolidate**: **link two roots of equal degree** (larger key under smaller) until all degrees differ, using an array indexed by degree; then scan the roots for the new min.

Decrease-key = cut + cascading cut: Lower x's key. If it is now smaller than its parent y, **cut** x: move it (with its subtree) to the root list and unmark it. Then: if y is **unmarked**, mark it and stop; if y is **already marked**, cut y too and repeat on y's parent — the **cascading cut** — until an unmarked node or a root is reached. In one line: **a non-root node may lose one child; losing a second gets it cut.**

Exam trap ✍: Why these heaps exist: a binary heap cannot **merge** faster than O(n). Binomial and Fibonacci heaps are pointer-based **mergeable heaps** (so are leftist and skew heaps).

5 Heap Costs Compared, and Why Dijkstra Wants a Fibonacci Heap

Worked example ✍ a cascading cut: In the tree on the previous sheet, 24 and 18 are marked. **Decrease 46 → 15:** 15 < parent 24 ⇒ cut 15 to the root list; 24 is marked ⇒ cut 24 (with 26) as well; its parent 7 is a root ⇒ stop. **Decrease 39 → 1:** 1 < 18 ⇒ cut 1; 18 is marked ⇒ cut 18; its parent 3 is a root ⇒ stop; **H.min = 1**. Each cut is O(1); the cascade is paid for by the marks it clears.

Potential function: `Φ(H) = t(H) + 2·m(H)`: t = trees in the root list, m = marked nodes. Consolidation is paid for by the trees it removes; each cascading cut **unmarks** a node (−2) and adds a tree (+1), so it pays for itself ⇒ decrease-key is **O(1) amortized**.

Where "Fibonacci" comes from: Since a node loses at most one child before it is cut, a node of degree k roots a subtree of at least $\mathbf{F_{k+2} \geq \varphi^k}$ nodes ($\varphi \approx 1.618$). So the maximum degree is $\mathbf{D(n) \leq \lfloor \log_{\varphi} n \rfloor \approx 1.44 \log_2 n}$ — which makes extract-min O(log n) amortized.

Operation	Binary heap (worst)	Binomial heap (worst)	Fibonacci heap (amortized)
Insert	O(log n)	O(log n); O(1) amortized	**O(1)**
Find-min	O(1)	O(log n); O(1) with a min pointer	O(1)
Extract-min	O(log n)	O(log n)	O(log n)
Union / merge	O(n)	**O(log n)**	**O(1)**
Decrease-key	O(log n)	O(log n)	**O(1)**
Delete (decrease to −∞, extract)	O(log n)	O(log n)	O(log n)

Dijkstra / Prim: count the heap calls ✍: On a graph with V vertices and E edges both algorithms do **V inserts**, **V extract-mins** and up to **E decrease-keys**. **Array (no heap):** V × O(V) ⇒ **O(V²)** — best for dense graphs. **Binary heap:** (V + E) × O(log V) ⇒ **O((V + E) log V)**. **Fibonacci heap:** V × O(log V) + E × O(1) ⇒ **O(E + V log V)**. The gain is in the **E decrease-keys**: O(log V) → O(1). (Algorithms: graph-algorithms lesson.)

Exam trap ✍ two: **1.** Fibonacci bounds are **amortized**: one extract-min can really cost **O(n)** (n roots after n inserts), one decrease-key O(log n). **2.** Extract-min is **never** O(1) here.

Memory trick: "**Fibonacci is lazy: everything is O(1) except taking the minimum out.**" Insert, union, find-min, decrease-key — O(1); extract-min and delete — O(log n), because that is when the lazy heap tidies its root list. And "**lose one child, get marked; lose two, get cut.**" In practice a binary heap is often faster.

6 B-Trees: Order m, Minimum Degree t, and the Height Bound

Definition: A **B-tree** (Bayer & McCreight, 1972) is a **balanced multiway search tree** designed for disks: one node = one disk block, holding many sorted keys. A node with **k keys has k + 1 children**, and the keys separate the key ranges of the children. **All leaves are at the same level**, so every search reads the same number of blocks. Because the fan-out is large, the height is tiny, and search, insert and delete are all **O(log n)**.

Order m (Knuth) — the exam convention: **1.** Every node has **at most m children** ⇒ at most **m − 1 keys**.
2. Every internal node except the root has **at least ⌈m/2⌉ children** ⇒ at least **⌈m/2⌉ − 1 keys** (non-root leaves keep the same key minimum).
3. The root has **at least 2 children** unless it is a leaf ⇒ at least 1 key.
4. All leaves on the same level.

Minimum degree t (CLRS): $t \geq 2$. Every non-root node has **t − 1 to 2t − 1 keys**, i.e. **t to 2t children** if internal. The root has at least 1 key. A node with 2t − 1 keys is **full**.
Translation: order m = 2t, so CLRS trees always have an even order. t = 2 ⇒ order 4 ⇒ the **2-3-4 tree**.

Order m	Max children	Max keys	Min children (non-root)	Min keys (non-root)	Also called
3	3	2	2	1	2-3 tree
4	4	3	2	1	2-3-4 tree (CLRS t = 2)
5	**5**	**4**	**3**	**2**	the usual exam example (next sheet)
m	m	m − 1	⌈m/2⌉	⌈m/2⌉ − 1	general rule

Height bound ✍ derivation: Fewest keys for height h (CLRS): root 1 key, then 2 nodes, 2t, $2t^2$, … , $2t^{h-1}$ nodes with t − 1 keys each:
`n ≥ 1 + (t-1)·Σ 2t^(i-1) = 2t^h - 1`
⇒ **h ≤ log_t((n + 1)/2)**. In order-m form: `h ≤ log_⌈m/2⌉((n+1)/2)`. Most keys at height h: every node full ⇒ `m^(h+1) - 1`.

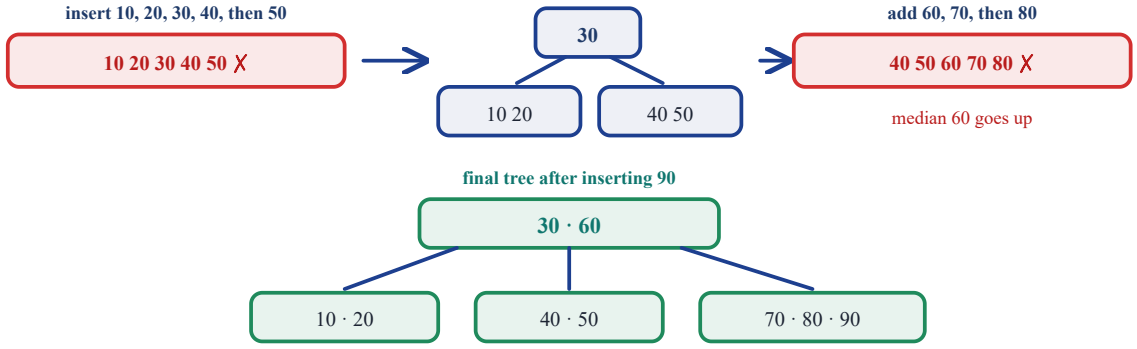
Cost of a search: Descend from the root, choosing the child between the two keys that bracket the target. **Disk reads = O(h) = O(log_t n)**. CPU work is O(t) per node by linear scan (O(log t) with binary search) ⇒ **O(t log_t n)** CPU in total. Disk reads dominate, which is why t is chosen so one node fills one block.

Exam trap ✍ which "order"?: "Order" in an MCQ almost always means **Knuth's maximum number of children**: order 5 ⇒ max **4** keys, min **2** keys in a non-root node. Do not answer "5 keys". If the question gives a **minimum degree t** instead, use t − 1 … 2t − 1 keys. And the minimum is ⌈m/2⌉ children — so ⌈m/2⌉ − 1 **keys**.

7 B-Tree Insertion (Split) and Deletion (Borrow / Merge)

Insertion: Search down to the correct **leaf** and insert the key in sorted position. If the leaf now has **m keys** (overflow), **split it**: the **median key moves up** into the parent, the keys left of it form one node and those right of it another. If the parent overflows, split it too. A split of the root makes a **new root** — **the tree grows only at the root**, which is why all leaves stay on one level.

Deletion: **1.** Key in a leaf with more than the minimum ⇒ just remove it. **2.** Key in an internal node ⇒ replace it by its **in-order predecessor** (largest key of the left subtree) or successor, then delete that key from its leaf. **3.** A leaf **underflows** (below $\lceil m/2 \rceil - 1$ keys) ⇒ **borrow** from an adjacent sibling that has spare keys (rotate through the parent), else **merge** with a sibling plus the parent's separator key. A merge can make the parent underflow; if the root empties, the tree **shrinks by one level**.



order-5 insertion of 10, 20, …, 90, step by step

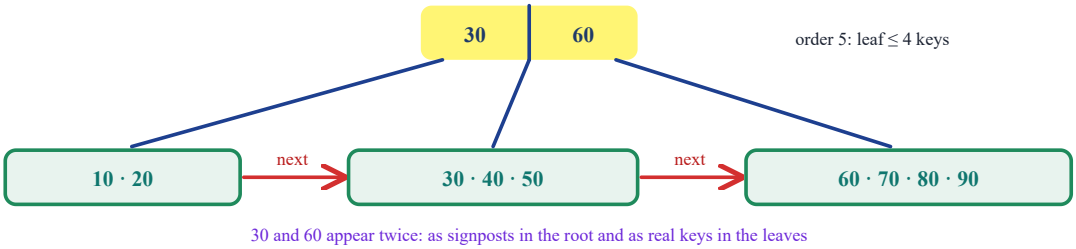
Worked deletions on that tree ✍️ order 5, min 2 keys: **Delete 40 (borrow):** leaf [40, 50] → [50] underflows. Left sibling [10, 20] has only the minimum, right sibling [70, 80, 90] has a spare ⇒ **rotate**: separator 60 comes down, 70 goes up. Result: root [30 | 70], leaves [10, 20] [50, 60] [80, 90].
Delete 20 from the original tree (merge): [10, 20] → [10] underflows; its only sibling [40, 50] has no spare ⇒ **merge** [10] + separator 30 + [40, 50] = [10, 30, 40, 50]. Root becomes [60] with leaves [10, 30, 40, 50] and [70, 80, 90] — still valid (root ≥ 1 key).

Memory trick: "**Too full? Split and push the middle up. Too empty? Borrow from a rich sibling, else merge and pull the separator down.**" Insert pushes keys **up**; delete pulls keys **down**. Height changes only at the root, in both directions.

Exam trap ✍️: **1.** A B-tree never grows at the leaves — new levels appear **above** the old root. **2.** With an **even** order the overflowing node has an even count, so there are two middles; textbooks differ (left- or right-biased) — state your choice. **3.** CLRS inserts in **one pass**, splitting every full node on the way down, so it never has to back up.

8 The B+ Tree, Database Indexes, and the Big Comparison

Definition: A **B+ tree** is a B-tree variant in which **all records (or record pointers) live in the leaves**; internal nodes hold only **copies of keys** as signposts, and the leaves are **linked in key order** (the sequence set).



the same nine keys as a correct two-level B+ tree: one root, three linked leaves

Point	B-tree	B+ tree
Where data / record pointers sit	in **every** node	**only in the leaves**
Keys stored	each key exactly once	separator keys repeated in internal nodes
Search	may stop early at an internal node	always ends at a leaf; higher fan-out ⇒ shorter tree
Range query / full scan	in-order traversal across levels	**O(log n + k)** along the leaf chain
Leaf split	median **moves** up	median **copied** up (stays in the leaf); internal splits still move it

Fan-out arithmetic ✍️: 4 KB page, 8-byte key + 8-byte child pointer ⇒ about **4096 ÷ 16 = 256** children per node. Three levels reach $256^3 = 16{,}777{,}216$ leaves. Root cached in RAM ⇒ any of ~16 million keys in **2–3 disk reads**.

Where they are used: **Databases:** MySQL InnoDB, PostgreSQL, Oracle and SQL Server use B+ trees for ordinary indexes. **File systems:** NTFS directory indexes (B+ tree), ext4 directories (HTree, a hashed B-tree), XFS and Btrfs (B-trees). B+ tree vs hash index: see the database chapter.

Structure	Find-min / max	Search any key	Insert	Best for
Binary heap	O(1)	O(n)	O(log n)	priority queue, heapsort
Fibonacci heap	O(1)	O(n)	O(1) amortized	many decrease-keys (Dijkstra, Prim)
B-tree	O(log n)	O(log n)	O(log n)	disk-based ordered storage
B+ tree	O(log n)	O(log n)	O(log n)	DB indexes, range queries

Trap: a heap is not a search structure — only find-**min** is O(1); "O(1) search in a Fibonacci heap" is wrong.

Night-Before-the-Exam Sheet

Fact sheet:

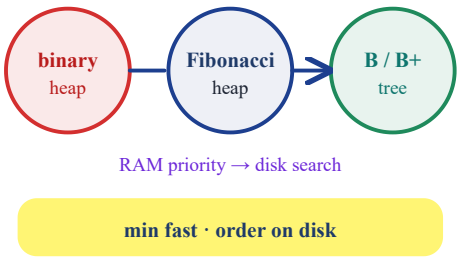
Heap = complete tree + heap order	Max-heap: max at root, min in a leaf
1-based: $2i$, $2i+1$, $\lfloor i/2 \rfloor$	0-based: $2i+1$, $2i+2$, $\lfloor (i-1)/2 \rfloor$
Height = $\lfloor \log_2 n \rfloor$	Leaves: $\lfloor n/2 \rfloor + 1 \ldots n$ ($\lceil n/2 \rceil$ of them)
Insert / extract / heapify: O(log n)	**Build-heap: O(n)**
Search in a heap: O(n)	Heapsort: O(n log n), in-place, unstable
d-ary: insert O(log_d n)	d-ary: extract O(d log_d n)
B_k: 2^k nodes, height k, C(k,i)	Binomial heap ↔ n in binary
Binomial union: O(log n)	Binary-heap union: O(n)
Fib: insert/union/find/decrease O(1)	Fib: extract-min/delete O(log n)
Φ = trees + 2 × marked	$D(n) \le \log_\varphi n$
Dijkstra + Fib: O(E + V log V)	Dijkstra + binary: O((V+E) log V)
Order m: ≤ m children, ≤ m−1 keys	Non-root ≥ $\lceil m/2 \rceil$ children
CLRS t: t−1 … 2t−1 keys	$h \le \log_t((n+1)/2)$
B+ leaves linked, data in leaves	B+ range: O(log n + k)

Top 12 one-liners for MCQs

1. Heap order says nothing about left vs right
2. Build-heap starts at $\lfloor n/2 \rfloor$, goes down to 1
3. $\Sigma\, h/2^h = 2 \Rightarrow$ build-heap is linear
4. Top-k with a size-k heap: O(n log k)
5. Mergeable heaps exist because of **union**
6. Cascading cut: second lost child ⇒ cut
7. Fibonacci bounds are **amortized**
8. Order 5: keys 2…4, children 3…5
9. Order 3 = 2-3 tree; order 4 = 2-3-4 tree
10. B-tree grows and shrinks at the **root**
11. Overflow ⇒ split; underflow ⇒ borrow / merge
12. B+ leaf split copies the key up

Top traps: $2i+1$ vs $2i$ (check the indexing) · heapify O(log n) vs build-heap O(n) · "O(1) search" in any heap · Fibonacci extract-min "O(1)" · amortized taken as worst case · order m giving "m keys" · min keys = $\lceil m/2 \rceil - 1$, not $\lceil m/2 \rceil$ · a B-tree "growing at the leaves".

All tricks together: Double to go down, halve to go up · a binomial heap is n in binary, union is addition · Fibonacci is lazy: all O(1) except taking the min out · lose one child, get marked; lose two, get cut · too full: split and push up; too empty: borrow, else merge.



two jobs: fast minimum, few disk reads

✓ Practice MCQs (exam pattern, 10 marks)

1. In a max-heap, the **largest** element is stored:
(a) At the leftmost leaf (b) At the last array position
(c) In any one of the leaves (d) At the root

2. Root at index 1: the children of node i are at:
(a) 2i and 2i + 1 (b) 2i + 1 and 2i + 2
(c) i + 1 and i + 2 (d) i/2 and i/2 + 1

3. Max-heap `[50, 30, 40, 10, 20]`; insert 45 with sift-up. The array becomes:
(a) [50,45,30,10,20,40] (b) [50,30,45,10,20,40]
(c) [45,30,50,10,20,40] (d) [50,30,40,10,20,45]

4. Building a binary heap from n unsorted keys bottom-up takes:
(a) O(log n) (b) O(n log n)
(c) O(n) (d) $O(n^2)$

5. Amortized cost of **decrease-key** in a Fibonacci heap:
(a) O(log n) (b) O(n)
(c) O(1) (d) O(log log n)

6. Dijkstra's algorithm with a Fibonacci heap runs in:
(a) $O(V^2)$ (b) O(E + V log V)
(c) O((V + E) log V) (d) O(E log E)

7. B-tree of order m = 5: max keys in a node and min keys in a non-root node?
(a) 5 and 3 (b) 4 and 3
(c) 5 and 2 (d) 4 and 2

8. How does the height of a B-tree increase?
(a) When the root splits (b) When a leaf splits
(c) When two leaves merge (d) When a key is borrowed

9. A binomial heap with 13 nodes contains the trees:
(a) B_3, B_1, B_0 (b) B_2, B_1, B_0
(c) B_3, B_2, B_1 (d) B_3, B_2, B_0

10. Why are B+ trees preferred over B-trees for database range queries?
(a) They are binary trees (b) All keys are in linked leaves
(c) They need no internal nodes (d) They store data in the root

Answer key → 1-d · 2-a · 3-b · 4-c · 5-c · 6-b · 7-d · 8-a · 9-d · 10-b

Written question: (a) Show that BUILD-MAX-HEAP runs in O(n). (b) Insert 10, 20, 30, 40, 50, 60, 70, 80, 90 into an empty B-tree of **order 5**, showing each split, then delete 40. (c) State the amortized costs of a Fibonacci heap and explain why Dijkstra then runs in O(E + V log V). 10 marks

1. **(a) Build-heap (3)** — heapify i = ⌊n/2⌋ … 1; a node of height h costs O(h) and there are ≤ ⌈n/2^(h+1)⌉ of them; Σ ⌈n/2^(h+1)⌉·O(h) = O(n Σ h/2^h) = O(2n) = O(n). 3
2. **(b) Splits (3)** — order 5 ⇒ max 4 keys, min 2. 10–40 fill one leaf; 50 overflows ⇒ median 30 up: [30] / [10 20] [40 50]. 60, 70 ⇒ [40 50 60 70]; 80 overflows ⇒ 60 up: [30 60] / [10 20] [40 50] [70 80]; 90 ⇒ [70 80 90]. 3
3. **(b) Delete 40 (2)** — [50] underflows; left sibling at minimum, right sibling [70 80 90] has a spare ⇒ borrow: 60 down, 70 up ⇒ [30 70] / [10 20] [50 60] [80 90]. 2
4. **(c) Fibonacci heap (2)** — insert, union, find-min, decrease-key O(1); extract-min, delete O(log n) amortized. Dijkstra = V extract-min + E decrease-key = O(V log V + E). 2

Before the exam ✓

- ✓ Can write both index formula sets
- ✓ Can sift-up / sift-down an array by hand
- ✓ Can prove build-heap is O(n)
- ✓ Can fill the three-heap cost table
- ✓ Can state order-m min/max keys
- ✓ Can split, borrow and merge in a B-tree